Universidad Nacional Autónoma de México
Facultad de Ingeniería

Criptografía

Cifrado con Curvas Elípticas (ECC) y ECIES

Silverio Martínez Andrés

PROFESORA: DRA. Rocío Alejandra Aldeco Pérez

Semestre 2026-1

Grupo: 1

Fecha de entrega: 23 de octubre de 2025

Cifrado con Curvas Elípticas (ECC) y ECIES

a) ¿Cómo funciona ECIES?

ECIES (Elliptic Curve Integrated Encryption Scheme) es un esquema de cifrado híbrido, lo que significa que combina la criptografía asimétrica (de clave pública/privada) con la criptografía simétrica (de clave única) para aprovechar ambos cifrados.

La principal ventaja de la Criptografía de Curva Elíptica (ECC) es que ofrece la misma seguridad que otros sistemas como RSA, pero con llaves mucho más pequeñas y rápidas de procesar.

El funcionamiento de ECIES es el siguiente:

1. **Acuerdo de Clave:** Para enviar un mensaje cifrado, primero se necesita la clave pública del receptor. El emisor genera un par de llaves efímero (temporal) propio. Usando su clave privada temporal y la clave pública del receptor, ambas partes utilizan el protocolo ECDH (Elliptic Curve Diffie-Hellman) para derivar un secreto compartido sin que este viaje por la red.
2. **Derivación de Claves Simétricas:** Este secreto compartido no se usa directamente para cifrar. En su lugar, se pasa a través de una Función de Derivación de Clave (KDF). Esta función genera dos claves simétricas distintas y seguras: una para el cifrado y otra para la autenticación.
3. **Cifrado del Mensaje:** El mensaje se cifra utilizando un algoritmo de cifrado simétrico rápido y robusto, como AES (Advanced Encryption Standard), usando la primera clave generada por la KDF.
4. **Autenticación del Mensaje:** Para asegurar que el mensaje no ha sido alterado, se crea un Código de Autenticación de Mensaje (MAC) sobre el texto ya cifrado. Esto se hace con la segunda clave generada por la KDF.
5. **Envío:** Finalmente, el emisor envía al receptor un paquete que contiene: su clave pública temporal, el mensaje cifrado y el MAC.
6. **Descifrado:** El receptor utiliza su clave privada y la clave pública temporal del emisor para derivar el mismo secreto compartido. Con él, genera las mismas claves de cifrado y autenticación, verifica el MAC y, si es correcto, descifra el mensaje.

b) Cómo usaste la librería proporcionada

Para la parte práctica, utilicé la librería de Python eciespy, la cual simplifica enormemente la implementación de ECIES. Los pasos que seguí en el código fueron:

1. **Instalación**: Primero, instalé la librería en mi entorno de Python usando pip: pip install eciespy y demás necesarios para llevar a cabo la actividad.
2. **Importación**: Importé las clases y funciones necesarias del módulo ecies.
3. **Generación de Llaves**: Utilicé la función ecies.utils.generate_eth_key() para crear un nuevo par de llaves (privada y pública) basadas en la curva secp256k1, que es la misma que se utiliza en criptomonedas como Ethereum.
4. **Cifrado**: Definí un mensaje secreto en formato de bytes. Luego, usé la función ecies.encrypt(), pasándole la clave pública (en formato hexadecimal) y el mensaje. Esta función se encargó de todo el proceso ECIES internamente y devolvió los datos cifrados.
5. **Descifrado**: Finalmente, para comprobar que el proceso funcionó, utilicé la función ecies.decrypt(), a la cual le pasé la clave privada (en formato hexadecimal) y los datos cifrados. Como resultado, obtuve el mensaje original.

c) Captura de pantalla de la ejecución

```
PS C:\Users\andik\Desktop> python3 ./ecis.py
Llave Privada (hex): 0x8182553aad3d71fcd1024465b750b256dfde8890783889dcaa564dc08022fa91
Llave Pública (hex): 0x4db060febad54975b67aae0562e39cdfe5b74a99db16e497d462eda184f917ddccda1a0aeddeafe64939f536361a8a50d2d
24cabd96df37809a60bcd993e956e

Secreto Original: b'Este es un mensaje ultra secreto'

Mensaje Cifrado: b'\x04^\x81D!\xb6\xd6\xd3\x8c\xa7\xd3m%\x1cl;Lz\x00X\xe9:\xd8\xf9\x90\x1d\xf5\xf3p\xffw\t\xf5\xae\xff\x85
\xe96!g\xa6\x95G\xacA[\xba\xfc\xad0>\x7f\x0f\x87\x07\xfb\x10\xb2\x94\xdb\x98\xcf\x7f\xc4qW\tR\xf4\xe5\x88\x14p\xb7\xf8\xc6
\xd6\x06\xbb\xd4{\xa8XXmf\xa4\xb9\x19K1\x8d\xd9\x12\x8d\x82\x8c%\xb0\xcb\xe2\xffO\xdf\x8dJC!/\x80cD8\xdfFo\x9a\xfe\xb5\x8d
\xb0\x84-\xb1\x8b\xcfM\\\xd1'

Mensaje Descifrado: b'Este es un mensaje ultra secreto'

¡Éxito! El mensaje descifrado coincide con el original.
```

d) Código documentado

```python
"""
Silverio Martínez Andrés
Materia: Criptografía
Ejemplo de uso de ECIES (Elliptic Curve Integrated Encryption Scheme)
"""

# Importar las librerías necesarias
import ecies
from eth_keys import keys
from coincurve.utils import get_valid_secret

def main():
    """
    Función principal que demuestra el ciclo completo de ECIES:
    1. Este geenera un par de llaves (pública y privada).
    2. Se define un mensaje secreto.
    3. Se cifra el mensaje con la llave pública.
    4. Se descifra el mensaje con la llave privada.
    5. Por último, se verifica que el mensaje original y el descifrado coincidan.
    """

    # 1. Generar un par de llaves ECC (estilo Ethereum, curva secp256k1)
    llave_privada = keys.PrivateKey(get_valid_secret())
    llave_privada_hex = llave_privada.to_hex()
    llave_publica_hex = llave_privada.public_key.to_hex()

    print(f"Llave Privada (hex): {llave_privada_hex}")
    print(f"Llave Pública (hex): {llave_publica_hex}\n")
```

```python
    # 2. Definir el mensaje secreto que queremos compartir
    secreto_original = b'Este es un mensaje ultra secreto'
    print(f"Secreto Original: {secreto_original}\n")

    # 3. Se cifra el mensaje usando la llave punlica del destinatario
    mensaje_cifrado = ecies.encrypt(llave_publica_hex, secreto_original)
    print(f"Mensaje Cifrado: {mensaje_cifrado}\n")

    # 4. Se descifra el mensaje usando la llave privada correspondiente
    mensaje_descifrado = ecies.decrypt(llave_privada_hex, mensaje_cifrado)
    print(f"Mensaje Descifrado: {mensaje_descifrado}\n")

    # 5. Verificación final
    if secreto_original == mensaje_descifrado:
        print("¡Éxito! El mensaje descifrado coincide con el original.")
    else:
        print("Error: El mensaje descifrado no coincide.")

if __name__ == "__main__":
    main()
```

e) Conclusiones

A través de esta pequeña actividad realizada en horario de clase, se ha podido comprobar el poder y la relativa sencillez de implementación de los esquemas de cifrado modernos como ECIES

La Criptografía de Curva Elíptica es una base criptográfica potente que permite alcanzar altos niveles de seguridad con llaves más cortas en comparación con algoritmos más antiguos como RSA, lo que se traduce en un menor costo computacional.

ECIES demuestra la eficacia de combinar la criptografía asimétrica y simétrica. Se utiliza la primera para el intercambio seguro de claves y la segunda para el cifrado masivo de datos, logrando un sistema seguro y eficiente.

Herramientas como eciespy son fundamentales para los desarrolladores, ya que abstraen la complejidad matemática subyacente y permiten implementar sistemas de cifrado robustos de manera rápida y con menor probabilidad de errores.

Esta tarea demuestra que, aunque los fundamentos matemáticos de la criptografía son complejos, su aplicación práctica está al alcance de los desarrolladores gracias a las herramientas de software disponibles hoy en día.